Unity Game Design Document

2D Tower Defense Game

공개 저장소 분석 기반 설계 문서

원본 저장소	EvanderChristianDumalang / Tower-Defense https://github.com/EvanderChristianDumalang/Tower-Defense
라이선스	MIT License — Copyright (c) 2022 Evander Christian Dumalang (저장소 README 기준: 코드에 한정. 에셋 출처 표기는 없음)
Unity 버전	2018.3.3f1
분량	스크립트 8개 / 748줄 · 씬 2개 · 프리팹 12개 · 스프라이트 15장 · 오디오 2개
문서 작성	원광대학교 게임콘텐츠학과「게임프로그래밍」
작성일	2026년

목 차

1. 문서 개요 4
1-1. 이 문서는 무엇인가 4
1-2. 왜 이 저장소인가 4
1-3. 읽는 순서 4
2. 게임 개요 6
2-1. 한 줄 요약 6
2-2. 기본 정보 6
2-3. 플레이어가 하는 일 — 그리고 하지 않는 일 6
3. 핵심 게임 루프 7
3-1. 한 판의 흐름 7
3-2. 매 프레임 일어나는 일 7
4. 조작 9
5. 레벨 설계 10
5-1. 두 레벨의 공통 규격 10
5-2. Level1 의 실제 좌표 11
5-3. 슬롯 배치의 의도 11
6. 적 12
6-1. 적 2종 12
6-2. 등장 방식 12
6-3. 적의 이동 — 네 방향 스냅 12
6-4. 체력과 사망 13
7. 타워 14
7-1. 타워 3종 14
7-2. 매 프레임 세 단계 14
7-3. 설치 — 드래그와 슬롯 15
7-4. 설치에 걸리는 제한 15
8. 발사체 16
8-1. 탄에는 수치가 없다 16
8-2. 반드시 명중한다 16
9. UI 와 승패 17
9-1. 화면에 표시되는 것 17
9-2. 승리 판정 17
9-3. 패배 판정 17
10. 오디오 19
11. 밸런스 분석 20
11-1. 타워 하나가 적 하나에게 넣는 피해 20
11-2. 그래서 타워 3종 중 1종이 나머지를 지배한다 21
11-3. 수업 교재로 쓰려면 — 조정 설계안 22
12. 기술 구조 24

12-1. 스크립트 8개 24
12-2. 잘 되어 있는 것 24
12-3. 고쳐야 할 구조 24
13. 에셋 26
14. 확인된 결함 14건 27
14-1. 재현 경로까지 확인한 것 27
14-2. 설정 실수 · 이식성 · 설계 28
15. 수업 활용 방안 29
15-1. 그대로 쓸 수 있는가 29
15-2. 실습 과제 4개 29
15-3. 코드 참고용으로 쓸 부분 29
15-4. 확장 로드맵 30
부록 A. 씬 · 프리팹 수치 전체 31
A-1. LevelManager 인스펙터 (두 씬 동일) 31
A-2. 타워 프리팹 31
A-3. 적 프리팹 31
A-4. 경로 좌표 32
부록 B. 함께 제공하는 자료 32

1. 문서 개요

1-1. 이 문서는 무엇인가

이 문서는 GitHub 공개 저장소 EvanderChristianDumalang / Tower-Defense 를 대상으로, 그 안에 실제로 구현되어 있는 내용을 Unity GDD 형식으로 다시 정리한 것이다. 기획서를 먼저 쓰고 만든 게임이 아니라, 이미 만들어진 게임을 읽어서 기획서를 역으로 복원한 문서다.

따라서 이 문서에 적힌 수치는 전부 .unity 와 .prefab 파일에서 직접 읽은 값이다. "그렇게 되어 있을 것이다" 라고 추정한 값은 없다. 추정한 부분은 그때마다 추정이라고 밝혔다.

1-2. 왜 이 저장소인가

2D 타워 디펜스 공개 저장소 여섯 개를 비교한 결과, 이 저장소는 둘 중 하나의 이유로 남았다.

- **LICENSE 파일이 실제로 있다.** 비교한 여섯 개 중 LICENSE 파일이 있는 것은 둘뿐이었다. GitHub 에서 라이선스가 없다는 것은 "마음대로 쓰라"가 아니라 "저작권 전면 보유"를 뜻한다. 수업 자료로 배포하려면 이 구분이 중요하다.
- **748 줄이다.** 한 학기에 학생이 전부 읽을 수 있는 분량이다. 1,600 줄짜리 저장소는 읽히지 않는다.

> **다만 이 문서를 읽기 전에 알아 둘 것**
>
> · MIT 는 코드에만 적용된다. 저장소 README 가 "All the code available under the MIT licenses" 라고 명시했다. 스프라이트와 오디오의 출처 표기는 없으므로, 에셋을 그대로 재배포하지 않는 편이 안전하다.
>
> · 이 프로젝트에는 골드·웨이브·업그레이드·판매가 없다. 타워 디펜스의 핵심 재미가 비어 있는 상태다. 11장에서 그 이유를 수치로 보인다.
>
> · 그래서 이 문서는 "이 게임의 설명서" 이면서 동시에 "무엇을 더 얹어야 수업 교재가 되는가" 의 설계서이기도 하다. 12장과 15장이 그 부분이다.

1-3. 읽는 순서

장	내용	누구에게 필요한가
2 ~ 4	게임 개요 · 핵심 루프 · 조작	이 게임이 무엇인지 알아야 하는 사람 전부
5 ~ 10	레벨 · 적 · 타워 · 탄 · UI · 오디오 의 실제 수치	직접 수정하거나 수치를 바꿔 볼 학생

장	내용	누구에게 필요한가
11 ~ 12	밸런스 분석 과 기술 구조	왜 이 게임이 재미가 없는지 알고 싶은 사람
13 ~ 14	에셋 목록 과 확인된 결함 14건	이 코드를 고쳐 쓸 사람
15	수업 활용 방안	교수자

2. 게임 개요

2-1. 한 줄 요약

정해진 길을 따라 걸어오는 적이 끝에 닿기 전에, 길가의 네 자리에 타워를 놓아 막는 2D 타워 디펜스.

2-2. 기본 정보

장르	2D 타워 디펜스 (고정 경로 · 고정 설치 위치)
시점	정사영 2D 탑다운. 카메라 orthographic size 5
플랫폼	PC (마우스 드래그 + 키보드 R). 터치 대응은 없음
1회 플레이 시간	약 80 ~ 110초 (적 15마리 × 스폰 간격 5초 + 경로 통과 시간)
레벨 수	2개 — Level1, Level2
진행 구조	Level1 → (버튼) → Level2. 저장·점수 기록 없음
승리 조건	적 15마리를 모두 처치
패배 조건	적 4마리가 경로 끝에 도달 (목숨 3)

2-3. 플레이어가 하는 일 — 그리고 하지 않는 일

한다	하지 않는다
화면 아래 UI 에서 타워를 끌어다 슬롯에 놓는다	타워를 사기 위해 돈을 모으지 않는다 (골드 없음)
어느 슬롯에 어떤 타워를 놓을지 고른다	타워를 강화하지 않는다 (업그레이드 없음)
R 키로 다시 시작한다	타워를 팔거나 옮기지 않는다 (철거 없음)
승리 후 다음 레벨 버튼을 누른다	웨이브를 기다리거나 다음 웨이브를 미리 보지 않는다 (웨이브 없음)

오른쪽 열이 이 프로젝트의 성격을 그대로 보여 준다. 조작은 완성되어 있고 전투도 동작하지만, 플레이어가 내려야 할 판단은 "네 칸을 채울까 말까" 하나로 줄어들어 있다. 11장에서 이것을 수치로 확인한다.

3. 핵심 게임 루프

3-1. 한 판의 흐름

```
게임 시작
  ├─ 목숨 3,  남은 적 15  표시
  ├─ 화면 아래에 타워 UI 버튼 3개 생성
  │
  ├─ [0초]  첫 적 등장  ──────────────┐
  │                                   │  5초마다 반복 (15마리)
  │    적:  웨이포인트를 차례로 따라 걷는다
  │    타워: 사거리 안의 최근접 적을 조준 → 회전 → 발사
  │    탄:   목표를 추적해 반드시 명중 → 피해 → 목표 체력 0이면 비활성
  │                                   │
  ├─ 적이 경로 끝에 닿으면  목숨 -1 ┘
  │
  ├─ 목숨 0        →  You Lose!
  └─ 적 15마리 소진 + 살아 있는 적 0  →  You Win!
```

3-2. 매 프레임 일어나는 일

이 프로젝트의 가장 큰 구조적 특징은, 타워와 적이 스스로 움직이지 않는다는 점이다. 여섯 개 스크립트의 Update() 가 전부 비어 있고, 모든 동작을 LevelManager.Update() 한 곳에서 호출한다.

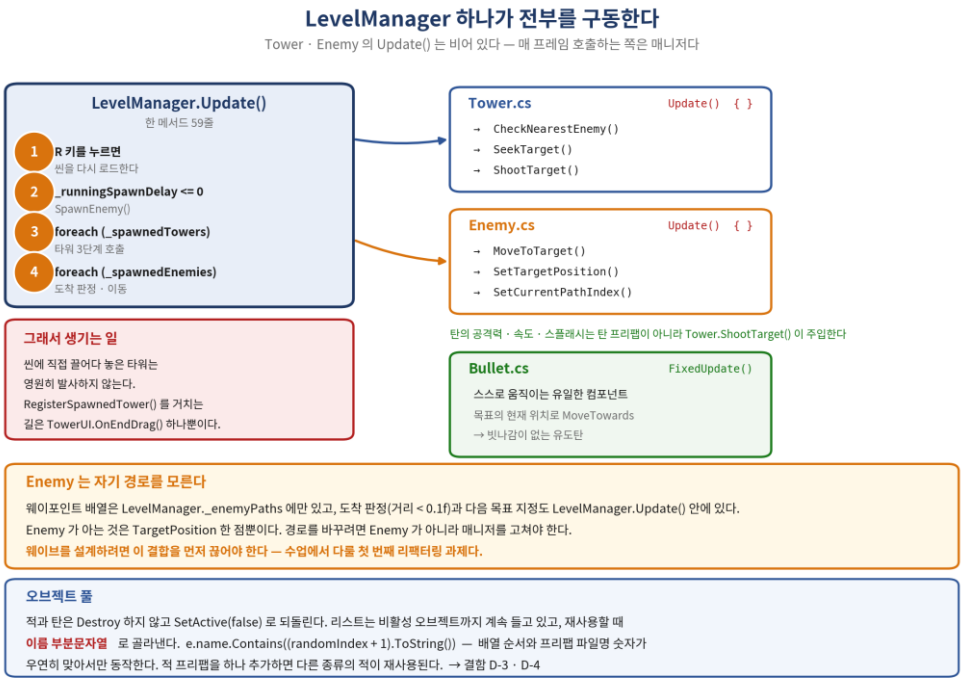


그림 3-1. LevelManager 가 매 프레임 호출하는 것들

여기서 따라오는 두 가지 결과

· 씬에 직접 끌어다 놓은 타워는 영원히 발사하지 않는다. 타워 목록에 등록되는 길은 TowerUI.OnEndDrag() 하나뿐이다.

· Enemy 는 자기 경로를 모른다. 웨이포인트도, 도착 판정도, 다음 목표 지정도 전부 LevelManager 안에 있다.

4. 조작

타워 설치	화면 아래 타워 아이콘을 마우스로 눌러 끈다 → 설치 슬롯 위에서 손을 뗀다. 슬롯 밖에서 손을 떼면 그 타워는 사라진다 (비용이 없으므로 손해도 없다).
재시작	R 키. 현재 씬을 다시 로드한다. 게임 오버 뒤에도 동작한다.
레벨 이동	결과 패널의 버튼. Play Again / Next / Prev 세 가지가 PlayAgain.cs 에 있다.
그 외	없음. 일시정지·속도 조절·타워 선택 해제·카메라 이동 모두 없다.

드래그가 동작하기 위한 조건 (학생이 자주 빠뜨리는 부분)

· 씬에 EventSystem 오브젝트가 있어야 한다. IBeginDragHandler / IDragHandler / IEndDragHandler 는 EventSystem 이 보내 주는 이벤트다.

· 타워 UI 오브젝트에 Raycast Target 이 켜진 Image 가 있어야 한다. 꺼져 있으면 클릭이 통과해 버려 드래그가 시작되지 않는다.

· 설치 슬롯(TowerPlacement)과 타워 중 한쪽에는 Rigidbody2D 가 있어야 트리거가 발생한다. Unity 의 2D 트리거 규칙이다.

· Tower 컴포넌트와 Collider2D 는 같은 게임오브젝트(루트)에 있어야 한다. 슬롯이 collision.GetComponent<Tower>() 로 찾기 때문이다.

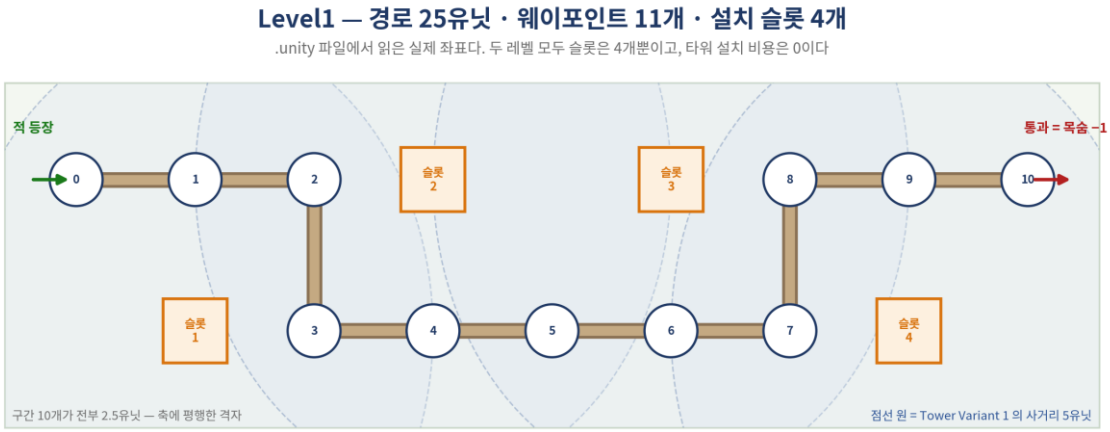
5. 레벨 설계

5-1. 두 레벨의 공통 규격

항목	Level1	Level2
웨이포인트 개수	11개	13개
경로 전체 길이	25.0 유닛	30.0 유닛
타워 설치 슬롯	4개	4개
스폰 간격 (_spawnDelay)	5초	5초
목숨 (_maxLives)	3	3
적 총 수 (_totalEnemy)	15	15
등장하는 적 종류	Enemy Variant 1, Enemy Variant 2	동일
설치할 수 있는 타워 종류	Tower Variant 1 · 2 · 3	동일
카메라 orthographic size	5	5

두 레벨의 차이는 경로의 모양과 길이뿐이다. 적의 수·종류·속도, 목숨, 타워 종류, 슬롯 개수가 전부 같다. 즉 Level2 는 "더 어려운 레벨" 이 아니라 "더 긴 레벨" 이다. 경로가 길면 타워가 쏠 시간이 늘어나므로 오히려 더 쉽다.

5-2. Level1 의 실제 좌표



Tower Variant 1 을 놓았을 때	경로 노출 길이	Enemy Variant 1 (체력 3 · 속도 2)		Enemy Variant 2 (체력 6 · 속도 1)	
슬롯 1 (−7.5, 0)	10.00 유닛	2발 / 4 피해	죽는다	4발 / 8 피해	죽는다
슬롯 2 (−2.5, 2.5)	11.83 유닛	2발 / 4 피해	죽는다	4발 / 8 피해	죽는다
슬롯 3 (2.5, 2.5)	11.83 유닛	2발 / 4 피해	죽는다	4발 / 8 피해	죽는다
슬롯 4 (7.5, 0)	10.00 유닛	2발 / 4 피해	죽는다	4발 / 8 피해	죽는다

Tower Variant 1 을 아무 슬롯 하나에만 놓아도 두 종류의 적이 모두 죽는다.
슬롯 4개를 전부 채우면 Enemy Variant 1 에 누적 16 피해가 들어간다 — 체력 3의 5배다. 비용이 0이므로 아끼는 이유도 없다.

그림 5-1. Level1 — .unity 파일에서 읽은 웨이포인트 · 슬롯 좌표와 사거리

Level1 의 경로는 모든 구간이 정확히 2.5유닛인 축 평행 격자다. 꺾이는 곳이 네 군데이고, 꺾임마다 적의 스프라이트 회전이 90도 단위로 바뀐다.

```
WP0  (-10.0,  2.5)      WP6  (  2.5,  0.0)
WP1  ( -7.5,  2.5)      WP7  (  5.0,  0.0)
WP2  ( -5.0,  2.5)      WP8  (  5.0,  2.5)
WP3  ( -5.0,  0.0)      WP9  (  7.5,  2.5)
WP4  ( -2.5,  0.0)      WP10 ( 10.0,  2.5)   ← 여기 닿으면 목숨 -1
WP5  (  0.0,  0.0)

설치 슬롯  (-7.5, 0.0)  (-2.5, 2.5)  ( 2.5, 2.5)  ( 7.5, 0.0)
```

5-3. 슬롯 배치의 의도

네 슬롯은 경로의 직선 구간을 하나씩 맡도록 놓여 있다. 슬롯 1과 4는 경로에서 2.5유닛 떨어진 바깥쪽, 슬롯 2와 3은 두 직선 사이의 안쪽이다. 안쪽 두 슬롯은 위아래 두 줄을 동시에 덮으므로 노출 구간이 더 길다 (11.83유닛 대 10.00유닛).

배치 자체는 합리적이다. 문제는 그 차이가 결과를 바꾸지 못한다는 점인데, 11장에서 다룬다.

6. 적

6-1. 적 2 종

프리팹	체력	이동 속도	Level1 통과 시간	역할
Enemy Variant 1	3	2 유닛/초	12.5초	빠르지만 약하다
Enemy Variant 2	6	1 유닛/초	25.0초	느리지만 두 배 단단하다
Enemy.prefab (기본)	1	1 유닛/초	—	두 씬 어디에도 쓰이지 않는다. 변형 프리팹의 원본 역할만 한다

6-2. 등장 방식

웨이브라는 개념이 없다. 5초마다 두 종류 중 하나를 무작위로 뽑아 한 마리씩 내보내고, 15마리를 다 내보내면 끝이다.

```
int randomIndex = Random.Range (0, _enemyPrefabs.Length);   // 0 또는 1
// → Enemy Variant 1 과 2 가 정확히 50 : 50 확률
```

따라서 이런 일이 일어난다

· 같은 레벨을 두 번 하면 적 구성이 다르다. 운이 나쁘면 Enemy Variant 2 (체력 6) 가 연달아 나오고, 운이 좋으면 약한 적만 나온다.

· 난이도 곡선이 없다. 1번째 적과 15번째 적이 완전히 같은 확률 분포에서 나온다. 뒤로 갈수록 어려워지는 느낌이 전혀 없다.

· 적 15마리의 총 체력은 평균 (3 + 6) / 2 × 15 = 67.5 이지만, 분산이 커서 45 ~ 90 사이에서 흔들린다.

6-3. 적의 이동 — 네 방향 스냅

적은 목표 웨이포인트를 향해 직선으로 이동하면서, 방향에 따라 스프라이트를 90도 단위로 회전시킨다. 경로가 축에 평행하므로 어긋나 보이지 않는다.

```
if (Mathf.Abs (distance.y) > Mathf.Abs (distance.x))
    → 위쪽이면  z =  90도,  아래쪽이면  z = -90도
else
    → 오른쪽이면 z =   0도,  왼쪽이면   z = 180도
```

회전할 때 체력바가 같이 돌아가면 안 되므로, 회전 직전에 체력바를 계층에서 떼어 냈다가 회전 뒤에 다시 붙인다. 의도는 맞지만 웨이포인트를 지날 때마다 트랜스폼 계층을 두 번 바꾸는 비싼 연산이다 (결함 D-

13).

6-4. 체력과 사망

- 체력은 OnEnable() 에서 최대치로 되돌아간다. Awake 나 Start 가 아닌 이유는 적이 오브젝트 풀에서 재사용되기 때문이다.
- 피격될 때마다 hit-enemy 효과음이 난다. 체력이 0 이하가 되면 오브젝트를 비활성하고 enemy-die 효과음을 낸다. 죽는 프레임에는 두 소리가 겹쳐 난다.
- 죽은 적은 Destroy 되지 않고 죽은 자리에 비활성 상태로 남는다. 이것이 결함 D-2 의 원인이 된다.

7. 타워

7-1. 타워 3 종

프리팹	공격력	사거리	발사 간격	초당 피해	스플래시	쓰는 탄
Tower Variant 1	2	5	3초	0.67	없음	Bullet Variant 1
Tower Variant 2	1	8	4초	0.25	없음	Bullet Variant 1
Tower Variant 3	1	4	5초	0.20	반지름 2	Bullet Variant 2
Tower.prefab (기본)	1	5	5초	0.20	없음	지정되지 않음

기본 Tower.prefab 은 두 씬 어디에도 쓰이지 않는다. 세 변형 프리팹의 원본 역할만 한다. 탄 속도는 네 개 모두 4 유닛/초로 같다.

7-2. 매 프레임 세 단계

단계	하는 일	세부
1. CheckNearestEnemy	사거리 안에서 가장 가까운 적을 고른다	이미 목표가 있고 아직 사거리 안이면 그대로 둔다. 목표를 놓치면 전체 목록을 다시 훑는다.
2. SeekTarget	포탑 머리를 목표 쪽으로 돌린다	회전 속도 180도/초. 스프라이트가 위쪽(+Y)을 보도록 그려져 있어 각도에 −90도 보정이 들어간다.
3. ShootTarget	조준이 끝났고 쿨타임이 지났으면 쏜다	조준 오차 10도 이내여야 발사한다. 쿨타임 초기값이 0이라 목표를 잡는 즉시 첫 발이 나간다.

조준 지연이 실제로 영향을 준다

· 목표를 새로 잡으면 포탑이 최대 180도 돌아야 하므로 최대 1초가 걸린다. 그 사이에는 쿨타임이 끝나도 발사하지 않는다.

· 즉 "발사 간격 3초" 는 이상적인 값이고, 목표가 자주 바뀌면 실질 발사 빈도는 그보다 낮아진다.

7-3. 설치 — 드래그와 슬롯

설치는 세 스크립트가 맞물려 동작한다.

```
TowerUI.OnBeginDrag()   타워 프리팹을 Instantiate 해서 마우스에 붙인다
      ↓ (드래그 중)
TowerUI.OnDrag()        마우스 화면 좌표 → 월드 좌표로 변환해 따라다니게 한다
      ↓ (슬롯 위에 겹치면)
TowerPlacement.OnTriggerEnter2D()   슬롯이 자기 좌표를 타워에게 알려 준다
      ↓ (손을 뗐을 때)
TowerUI.OnEndDrag()
   PlacePosition == null  →  Destroy        (슬롯 밖이었다)
   PlacePosition != null  →  LockPlacement()  슬롯 좌표에 고정
                          →  LevelManager.RegisterSpawnedTower()
                             ★ 이 등록을 거쳐야만 타워가 발사한다
```

"아직 어떤 슬롯에도 올라가지 않았다"를 Vector2? (nullable) 의 null 로 표현한 점은 잘 쓴 부분이다.

Vector2.zero 를 "없음"으로 쓰면 원점에 있는 슬롯과 구별할 수 없다.

7-4. 설치에 걸리는 제한

비용	없다. 드래그할 때마다 새 타워가 생성된다. 같은 버튼을 몇 번이든 다시 끌 수 있다.
개수 제한	설치 슬롯 개수뿐. 레벨당 4개다.
철거 / 판매	없다. 한 번 놓으면 되돌릴 수 없다.
같은 자리에 두 개	원칙적으로는 막혀 있다. 그런데 결함 D-1 때문에 실제로는 가능하다 (14장).

8. 발사체

8-1. 탄에는 수치가 없다

Bullet Variant 1 과 Bullet Variant 2 는 이름과 스프라이트만 다르다. 공격력·속도·스플래시 반지름은 프리팹에 들어 있지 않고, 발사하는 순간 타워가 주입한다.

```
// Tower.ShootTarget() 안
Bullet bullet = LevelManager.Instance.GetBulletFromPool (_bulletPrefab);
bullet.transform.position = transform.position;
bullet.SetProperties (_shootPower, _bulletSpeed, _bulletSplashRadius);  // ← 여기
bullet.SetTargetEnemy (_targetEnemy);
bullet.gameObject.SetActive (true);
```

즉 탄의 성능은 탄이 아니라 타워가 결정한다. 같은 Bullet Variant 1 이라도 Tower Variant 1 이 쏘면 피해 2, Tower Variant 2 가 쏘면 피해 1 이다. 수치를 바꾸려면 타워 프리팹을 고쳐야 한다.

8-2. 반드시 명중한다

탄은 매 FixedUpdate 마다 목표의 현재 위치로 MoveTowards 한다. 따라서 빗나감이 없다. 목표가 먼저 죽으면 탄은 스스로 비활성되어 풀로 돌아간다.

상황	동작
목표가 살아 있다	목표 쪽으로 이동 + 진행 방향으로 회전
목표가 비활성되었다	탄도 즉시 비활성. 목표 참조를 null 로 지운다
목표와 충돌했다 (스플래시 0)	목표에게만 피해
목표와 충돌했다 (스플래시 > 0)	LevelManager.ExplodeAt() 으로 반지름 안의 모든 적에게 피해
목표가 아닌 다른 적과 닿았다	아무 일도 없다. 자기 목표인지 확인한다

스플래시가 있는 것은 Tower Variant 3 (반지름 2) 하나뿐이다. 그런데 적이 5초 간격으로 한 마리씩 나오므로 반지름 2 안에 두 마리가 동시에 들어오는 일이 거의 없다. 광역 공격이 설계되어 있지만 발동할 상황이 만들어지지 않는다.

9. UI 와 승패

9-1. 화면에 표시되는 것

요소	내용	구현
Lives	"Lives: 3" — 남은 목숨	UnityEngine.UI.Text (레거시)
Total Enemy	"Total Enemy: 14" — 앞으로 더 나올 적 수	UnityEngine.UI.Text (레거시)
타워 선택 바	타워 아이콘 3개. 드래그해서 설치	Tower UI 프리팹을 _towerPrefabs 수만큼 복제
결과 패널	"You Win!" / "You Lose!" + 버튼	평소에는 비활성, 승패가 정해지면 활성
적 체력바	적 머리 위의 작은 막대	SpriteRenderer 의 size 를 직접 줄인다

Total Enemy 는 "남아 있는 적" 이 아니라 "앞으로 더 생성될 적" 이다. 첫 적이 나오는 순간 15 에서 14 로 바뀐다.

9-2. 승리 판정

```
// LevelManager.SpawnEnemy()  – 5초마다 호출되는 함수 안에 들어 있다
SetTotalEnemy (--_enemyCounter);
if (_enemyCounter < 0)
{
    bool isAllEnemyDestroyed =
        _spawnedEnemies.Find (e => e.gameObject.activeSelf) == null;
    if (isAllEnemyDestroyed) SetGameOver (true);
    return;
}
```

이 구조에서 생기는 일

· 판정이 스폰 함수 안에 있으므로 5초마다만 검사된다. 마지막 적이 죽어도 승리 화면은 최대 5초 뒤에 뜬다 (결함 D-6).

· 적 수는 정확히 15마리다. _enemyCounter 가 15에서 시작해 14 → 0 까지 내려가므로 15번 생성되고, 16번째 호출에서 −1 이 되어 판정으로 넘어간다. 오프바이원이 아니다.

9-3. 패배 판정

적이 마지막 웨이포인트에 닿으면 ReduceLives(1) 이 호출되고 그 적은 비활성된다. 목숨이 0 이 되는 즉시 패배 화면이 뜬다. 승리와 달리 지연이 없다.

목숨이 3이므로 4마리째가 통과하는 순간 패배다. 타워를 하나도 놓지 않으면 Level1 기준 27~40초 사이에 끝난다.

10. 오디오

효과음 두 개가 전부다. 배경음악은 없다.

클립	언제 나는가	호출 위치
hit-enemy.ogg	적이 피해를 받을 때마다	Enemy.ReduceEnemyHealth()
enemy-die.ogg	적의 체력이 0 이 되었을 때	Enemy.ReduceEnemyHealth()

AudioPlayer 는 클립 이름 문자열로 찾는다. PlaySFX("hit-enemy") 처럼 호출하면 등록된 목록에서 이름이 같은 클립을 찾아 PlayOneShot 한다.

문자열 키의 위험 — 조용히 실패한다

· 인스펙터에서 클립을 등록하지 않았거나 파일 이름이 바뀌면, 오류도 경고도 없이 그냥 소리만 나지 않는다. 찾지 못하면 return 할 뿐이기 때문이다.

· 학생이 "소리가 안 나요" 라고 할 때 가장 먼저 확인할 곳이다. Console 에는 아무것도 남지 않는다.

· 개선 방향: enum 이나 ScriptableObject 로 바꾸면 컴파일 시점에 오타를 잡을 수 있다.

11. 밸런스 분석

이 장은 이 문서에서 가장 중요한 부분이다. 결론부터 적으면, 이 게임의 수치는 설계되지 않았다.

11-1. 타워 하나가 적 하나에게 넣는 피해

타워를 슬롯에 놓으면, 적은 그 타워의 사거리 원을 지나가는 동안만 맞는다. 그 구간의 길이를 경로를 따라 실제로 적분해 구하고, 적의 속도로 나누면 노출 시간이 나온다. 쿨타임 초기값이 0이므로 발사 횟수는 ⌊노출 시간 ÷ 발사 간격⌋ + 1 이다.

슬롯 (Tower Variant 1)	노출 길이	Enemy Variant 1 (체력 3)	Enemy Variant 2 (체력 6)
슬롯 1 (−7.5, 0)	10.00 유닛	4.9초 → 2발 → 피해 4 → 죽는다	10.0초 → 4발 → 피해 8 → 죽는다
슬롯 2 (−2.5, 2.5)	11.83 유닛	5.9초 → 2발 → 피해 4 → 죽는다	11.8초 → 4발 → 피해 8 → 죽는다
슬롯 3 (2.5, 2.5)	11.83 유닛	5.9초 → 2발 → 피해 4 → 죽는다	11.8초 → 4발 → 피해 8 → 죽는다
슬롯 4 (7.5, 0)	10.00 유닛	4.9초 → 2발 → 피해 4 → 죽는다	10.0초 → 4발 → 피해 8 → 죽는다

네 슬롯 모두 같은 결과다. Tower Variant 1 하나를 아무 데나 놓기만 해도 두 종류의 적이 모두 죽는다. 슬롯 위치를 고르는 의미가 없다.

11-2. 그래서 타워 3 종 중 1 종이 나머지를 지배한다

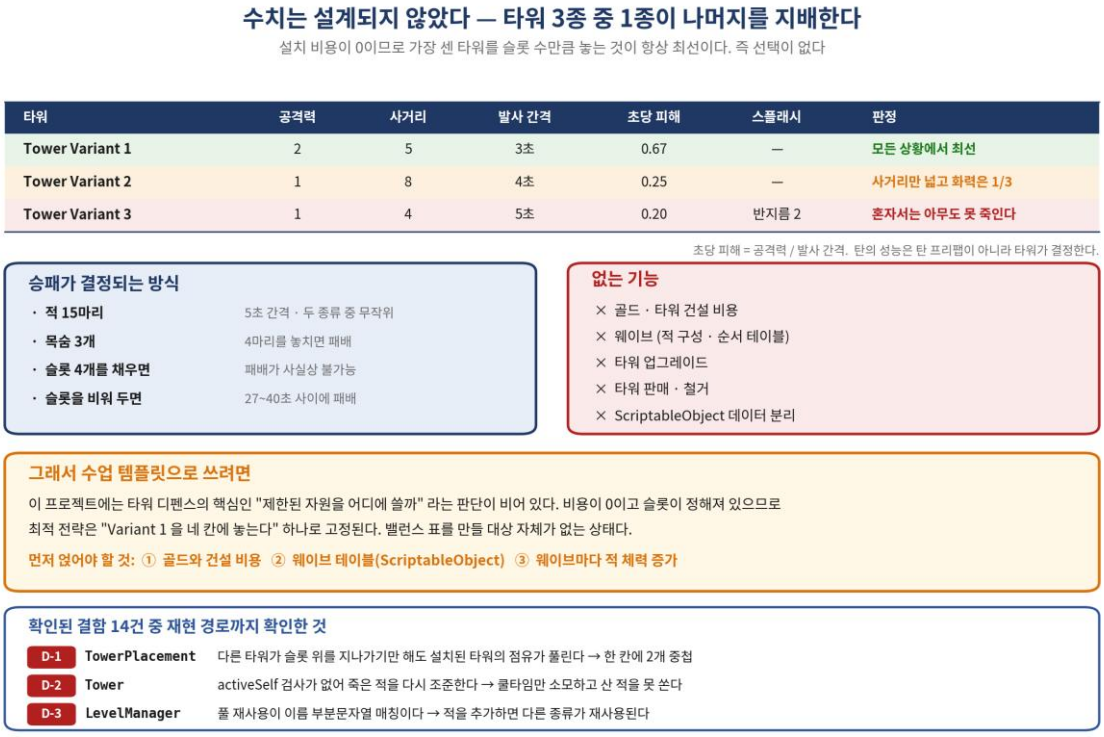


그림 11-1. 타워 성능 비교와 승패 구조

타워	초당 피해	혼자서 Enemy Variant 1 (체력 3) 을 죽이는가
Tower Variant 1	0.67	죽인다 (2발 / 피해 4)
Tower Variant 2	0.25	슬롯 2·3 에서만 죽인다 (3발 / 피해 3)
Tower Variant 3	0.20	죽이지 못한다 (1발 / 피해 1)

지배 전략이 하나로 고정된다

· 설치 비용이 0이므로 비싼 타워를 아낄 이유가 없다. 따라서 가장 센 타워를 슬롯 수만큼 놓는 것이 언제나 최적이다.

· 최적 전략 = "Tower Variant 1 을 네 칸 모두에 놓는다". 이 경우 Enemy Variant 1 에 누적 16 피해가 들어간다 — 체력 3의 다섯 배다.

· Tower Variant 2 와 3 을 고를 이유가 존재하지 않는다. 사거리가 넓거나 광역이라는 장점이 비용 절약으로 보상받지 못하기 때문이다.

· 반대로 슬롯을 하나도 채우지 않으면 27~40초 사이에 반드시 패배한다. 즉 이 게임의 모든 결과가 "네 칸을 채웠는가" 하나로 결정된다.

11-3. 수업 교재로 쓰려면 — 조정 설계안

아래는 측정값이 아니라 제안이다. 이 프로젝트에 "제한된 자원을 어디에 쓸까" 라는 판단을 만들어 넣기 위한 최소 변경안이다.

(1) 골드와 건설 비용을 넣는다

항목	제안 값	근거
시작 골드	120	Tower Variant 1 하나(70) + Variant 2 하나(50) 를 겨우 살 수 있는 액수
적 처치 보상	10	15마리를 다 잡으면 150 — 중간에 한 번 더 증축할 수 있다
Tower Variant 1 가격	70	가장 센 타워를 가장 비싸게 — 지배 전략을 깬다
Tower Variant 2 가격	50	사거리 8의 넓은 커버를 싸게 — 초반 포석용
Tower Variant 3 가격	55	광역은 적이 몰릴 때만 값을 한다

(2) 슬롯을 4 개에서 8 개로 늘린다

슬롯이 4개뿐이면 골드를 넣어도 "네 칸을 어떤 순서로 채울까" 로만 끝난다. 8개로 늘리면 "넓게 깔까, 한곳에 몰아 둘까" 라는 배치 판단이 생긴다.

(3) 웨이브 테이블을 만든다

현재는 5초마다 무작위 한 마리다. 아래처럼 묶음으로 바꾸면 난이도 곡선과 준비 시간이 생기고, 적이 몰리므로 Tower Variant 3 의 광역이 처음으로 쓸모 있어진다.

웨이브	구성	적 간격	체력 배율	처치 보상 합계
1	Enemy Variant 1 × 6	1.2초	× 1.0	60
2	Enemy Variant 1 × 6, Variant 2 × 2	1.0초	× 1.2	80
3	Enemy Variant 1 × 8, Variant 2 × 4	0.9초	× 1.5	120
4	Enemy Variant 1 × 10, Variant 2 × 6	0.8초	× 1.9	160
5	Enemy Variant 1 × 12, Variant 2 × 10	0.7초	× 2.4	220

이 표를 그대로 믿지 말 것

· 위 수치는 "슬롯 8개를 Tower Variant 1 로 채웠을 때 5웨이브의 Enemy Variant 2 (체력 6 × 2.4 ≒ 15) 가 간신히 죽는다" 는 계산에서 역산한 출발점이다.

· 다만 적이 0.7초 간격으로 몰려 들어오면 타워 한 대가 한 번에 한 마리만 쏘므로, 실제 피해는 이 계산보다 크게 줄어든다. 이 효과는 수식으로 풀기 어렵다.

· 따라서 이 표는 플레이테스트의 출발점이지 정답이 아니다. 실제로 돌려 보고 체력 배율부터 조정하는 것이 맞다.

· 그리고 이것이야말로 수업에서 가르칠 내용이다 — 밸런스는 계산으로 시작해서 플레이테스트로 끝난다.

(4) 수치를 ScriptableObject 로 꺼낸다

지금은 모든 수치가 프리팹 안에 있다. 타워 세 종류의 공격력을 비교하려면 프리팹 세 개를 번갈아 열어야 한다. ScriptableObject 로 꺼내면 한 화면에서 비교하고 한 번에 고칠 수 있다. 위 (1)~(3) 을 손으로 조정하려면 사실상 필수다.

12. 기술 구조

12-1. 스크립트 8 개

파일	줄수	역할
LevelManager.cs	243	싱글턴 매니저. 스폰·타워 구동·적 이동·목숨·승패·오브젝트 풀을 혼자 다 한다
Tower.cs	135	최근접 적 조준 → 포탑 회전 → 발사. 동작은 매니저가 구동한다
Enemy.cs	100	체력·4방향 회전·체력바. 자기 경로는 모른다
Bullet.cs	86	목표를 추적해 반드시 명중하는 유도탄. 단일 / 광역 피해
TowerUI.cs	68	드래그로 타워를 꺼내 설치한다. 인터페이스 3종 구현
TowerPlacement.cs	46	맵에 깔린 설치 슬롯. 레벨당 4개
AudioPlayer.cs	47	문자열 키로 효과음을 찾아 재생하는 싱글턴
PlayAgain.cs	23	버튼 OnClick 용 씬 전환 3종. 씬 이름이 하드코딩되어 있다

12-2. 잘 되어 있는 것

- **오브젝트 풀.** 적과 탄을 Destroy 하지 않고 재사용한다. 2D 게임에서 GC 부담을 줄이는 기본 기법을 교재 분량 안에서 보여 준다.
- **nullable 로 "미정" 표현.** Vector2? 의 null 로 "아직 어떤 슬롯에도 안 올라갔다"를 표현한다. 매직 넘버를 쓰지 않은 좋은 예다.
- **드래그 인터페이스 3 종 구현.** IBeginDragHandler / IDragHandler / IEndDragHandler 를 제대로 나눠 썼다. Unity UI 이벤트 시스템 수업에 그대로 쓸 수 있다.
- **OnDrawGizmos 로 경로 시각화.** Play 를 누르지 않고도 씬 뷰에서 적의 경로가 cyan 선으로 보인다. 에디터 보조 도구의 가장 간단한 예다.

12-3. 고쳐야 할 구조

문제	지금	바꿀 방향
God class	LevelManager 243줄 / Update() 59줄 에 네 가지 책임이 섞여 있다	EnemySpawner · TowerController · GameState 세 개로 쪼갠다

문제	지금	바꿀 방향
데이터가 프리팹 안에	수치를 비교하려면 프리팹을 하나씩 열어야 한다	ScriptableObject 로 꺼낸다
경로가 매니저 소유	Enemy 가 자기 경로를 모른다	Path 컴포넌트를 만들고 Enemy 가 참조한다
문자열 매칭 풀	이름 부분문자열로 재사용 대상을 찾는다	프리팹별 큐(Dictionary)로 분리한다
레거시 API	FindObjectOfType · UnityEngine.UI.Text	FindFirstObjectByType · TextMeshPro

13. 에셋

종류	개수	내용
씬	2	Level1.unity, Level2.unity (둘 다 텍스트 직렬화 — 협업 가능)
프리팹	12	타워 4 (기본 + 변형 3) · 적 3 (기본 + 변형 2) · 탄 4 · 타워 UI 1
스프라이트	15	tile_grass, road_1~2, tower_1~3, tower_plate, tower_placement, enemy_1~2, bullet_1~2, panel_blue, panel_glass, box_white
오디오	2	hit-enemy.ogg, enemy-die.ogg
ScriptableObject	0	없다
에디터 확장	0	없다

에셋 재배포에 대한 주의

· LICENSE 파일의 MIT 는 저장소 README 가 "All the code" 라고 범위를 적어 두었다. 스프라이트와 오디오가 여기에 포함되는지는 문서상 분명하지 않다.

· 원저작자가 직접 그린 것인지 외부 무료 팩을 쓴 것인지 저장소 안에 표기가 없다. 출처를 확인할 방법이 없으므로, 수업 자료로 배포할 때는 코드만 가져오고 스프라이트는 직접 준비하는 편이 안전하다.

· 이것 자체가 수업에서 다룰 주제다 — 에셋 출처 표기가 왜 필요한가.

14. 확인된 결함 14 건

코드를 읽어 확인한 결함이다. 재현 경로까지 확인한 것과, 설정 실수로 조용히 깨지는 것을 나눴다. 각 항목의 상세 설명은 함께 제공하는 Doxygen 문서의 해당 멤버 페이지에 있다.

14-1. 재현 경로까지 확인한 것

번호	파일	내용과 재현
D-1	TowerPlacement.cs	OnTriggerExit2D 가 떠나는 콜라이더를 확인하지 않는다. 재현: ① 타워 A 를 슬롯에 설치한다 ② 타워 B 를 드래그해 그 슬롯 위를 지나가게만 한다 ③ B 가 슬롯을 벗어나는 순간 A 의 점유가 풀린다 ④ 타워 C 를 같은 자리에 또 놓을 수 있다 고치는 법: OnTriggerExit2D 에서 collision 이 _placedTower 와 같은지 비교한다 (한 줄)
D-2	Tower.cs	CheckNearestEnemy 의 탐색 루프에 activeSelf 검사가 없다. 죽은 적은 Destroy 되지 않고 죽은 자리에 남으므로, 사거리 안이면 타워가 시체를 다시 조준한다. 쏜 탄은 목표가 비활성인 것을 보고 즉시 사라지고 쿨타임만 소모된다 → 타워가 멈춘 것처럼 보인다
D-3	LevelManager.cs	풀 재사용이 e.name.Contains((randomIndex + 1).ToString()) 라는 부분 문자열 비교다. 지금 맞게 보이는 이유는 배열 0번 프리팹 이름이 "Enemy Variant 1", 1번이 "2" 라서일 뿐이다. 재현: 적 프리팹을 하나 추가하거나 배열 순서를 바꾸면 다른 종류의 적이 재사용된다
D-4	LevelManager.cs	GetBulletFromPool 의 b.name.Contains(prefab.name) 도 같은 방식이다. 이름이 "Bullet" 인 프리팹을 넘기면 "Bullet Variant 1(Clone)" 도 조건을 통과한다
D-5	LevelManager / Tower / Bullet	스폰과 발사는 Time.unscaledDeltaTime, 적 이동은 Time.deltaTime, 탄 이동은 FixedUpdate 기준이다. Time.timeScale = 0 으로 일시정지를 만들면 적과 탄은 멈추는데 스폰과 발사만 계속된다. 원본 주석의 "어떤 것에도 영향받지 않는다" 는 설명은 틀렸다

번호	파일	내용과 재현
D-6	LevelManager.cs	승리 판정이 SpawnEnemy() 안에 있어 5초마다만 검사된다 → 승리 화면이 최대 5초 늦게 뜬다

14-2. 설정 실수 · 이식성 · 설계

번호	파일	내용
D-7	TowerUI.cs	OnEndDrag 의 Destroy 분기에서 _currentSpawnedTower 를 null 로 되돌리지 않는다
D-8	TowerUI.cs	OnDrag 가 Camera.main 을 두 번 조회한다. 드래그 중 매 프레임 태그 검색이 일어난다
D-9	Tower / AudioPlayer	using System.Collections.Generic; 중복 (CS0105 경고)
D-10	6개 파일	빈 Start() / Update() 가 12개. Unity 는 빈 Update() 도 매 프레임 호출한다
D-11	LevelManager / AudioPlayer	FindObjectOfType 은 Unity 2023.1부터 deprecated → FindFirstObjectByType
D-12	LevelManager.cs	UI 가 레거시 UnityEngine.UI.Text. 현재 권장은 TextMeshPro
D-13	Enemy.cs	웨이포인트마다 체력바를 parent = null → parent = transform 으로 재부모화한다
D-14	프로젝트 전체	골드·웨이브·업그레이드·판매가 없다. 설치 상한은 슬롯 4개뿐

15. 수업 활용 방안

15-1. 그대로 쓸 수 있는가

그대로는 쓸 수 없다. 이유는 둘이다. 첫째, Unity 2018.3 프로젝트라 현재 에디터로 열면 업그레이드 과정을 한 번 거쳐야 하고 레거시 API 경고가 난다. 둘째, 11장에서 보인 대로 플레이어가 내릴 판단이 비어 있어 "타워 디펜스를 만든다" 는 수업 목표를 이 상태로는 달성할 수 없다.

대신 아래 네 가지 용도로는 바로 쓸 수 있다.

15-2. 실습 과제 4 개

과제	다루는 개념	왜 이 결함인가	난이도
D-1 을 고쳐라	트리거 콜백에서 상대를 확인하는 이유	재현 절차가 명확하고(타워를 슬롯 위로 지나가게 한다) 수정이 한 줄이다. 첫 과제로 가장 좋다	하
D-2 를 찾아라	증상과 원인의 거리 · Debug.Log 추적	증상은 "타워가 안 쏜다" 인데 원인은 "시체를 조준한다" 이다. 로그를 찍어 좁혀 가는 연습이 된다	중
D-3 을 재현하라	문자열 비교에 기댄 코드의 취약성	적 프리팹을 하나 추가하면 바로 깨진다. "우연히 동작하는 코드" 의 교과서적 사례다	중
골드와 웨이브를 넣어라	ScriptableObject · 데이터 주도 설계	11-3 의 설계안을 그대로 명세로 준다. 기말 과제 분량이다	상

15-3. 코드 참고용으로 쓸 부분

주제	볼 파일	볼 곳
오브젝트 풀의 가장 단순한 형태	LevelManager.cs	SpawnEnemy() · GetBulletFromPool()
Unity UI 드래그 이벤트 3종	TowerUI.cs	OnBeginDrag / OnDrag / OnEndDrag
화면 좌표 → 월드 좌표 변환	TowerUI.cs	OnDrag() 의 ScreenToWorldPoint
nullable 로 "미정" 표현하기	Tower.cs	Vector2? PlacePosition
씬 뷰 보조선 그리기	LevelManager.cs	OnDrawGizmos()

주제	볼 파일	볼 곳
2D 트리거로 영역 점유 판정	TowerPlacement.cs	OnTriggerEnter2D / OnTriggerExit2D

15-4. 확장 로드맵

단계	얹을 것	수업 주차 기준
1	ScriptableObject 로 타워·적 수치를 꺼낸다	데이터 주도 설계 주차
2	골드와 건설 비용을 넣는다 (11-3 표)	게임 경제 주차
3	웨이브 테이블과 체력 배율을 넣는다	난이도 곡선 주차
4	LevelManager 를 세 클래스로 분리한다	리팩터링 주차
5	타워 업그레이드와 판매를 넣는다	최종 프로젝트

마지막으로

· 이 저장소의 값어치는 완성도가 아니라 크기와 라이선스에 있다. 748줄은 학생이 전부 읽을 수 있고, MIT 는 배포할 수 있게 해 준다.

· 그리고 결함 14건은 흠이 아니라 자산이다. 교재용으로 일부러 심은 버그보다 실제 프로젝트의 버그가 훨씬 가르치기 좋다 — 학생이 "실제 코드는 이렇게 생겼구나" 를 알게 되기 때문이다.

부록 A. 씬 · 프리팹 수치 전체

아래 값은 모두 .unity 와 .prefab 파일에서 직접 읽었다.

A-1. LevelManager 인스펙터 (두 씬 동일)

_spawnDelay	5
_maxLives	3
_totalEnemy	15
_towerPrefabs	Tower Variant 1, Tower Variant 2, Tower Variant 3
_enemyPrefabs	Enemy Variant 1, Enemy Variant 2
_enemyPaths	Level1 = 11개, Level2 = 13개

A-2. 타워 프리팹

프리팹	공격력	사거리	발사 간격	탄속	스플래시 반지름
Tower.prefab	1	5	5	4	0
Tower Variant 1	2	5	3	4	0
Tower Variant 2	1	8	4	4	0
Tower Variant 3	1	4	5	4	2

필드 이름은 차례로 _shootPower · _shootDistance · _shootDelay · _bulletSpeed · _bulletSplashRadius 이다.

A-3. 적 프리팹

프리팹	_maxHealth	_moveSpeed	씬에서 쓰이는가
Enemy.prefab	1	1	아니오
Enemy Variant 1	3	2	예
Enemy Variant 2	6	1	예

A-4. 경로 좌표

```
Level1  (11점 / 전체 25.0유닛 / 구간 10개가 전부 2.5유닛)
  (-10.0, 2.5) (-7.5, 2.5) (-5.0, 2.5) (-5.0, 0.0) (-2.5, 0.0) (0.0, 0.0)
  ( 2.5, 0.0) ( 5.0, 0.0) ( 5.0, 2.5) ( 7.5, 2.5) (10.0, 2.5)


Level2  (13점 / 전체 30.0유닛)
  (-10.0, 3.48) (-7.5, 3.48) (-5.0, 3.48) (-5.0, 0.98) (-2.44, 0.98)
  (-2.44,-1.52) ( 0.02,-1.52) ( 2.50,-1.55) ( 2.50, 0.95) ( 5.00, 0.98)
  ( 5.00, 3.48) ( 7.50, 3.48) (10.00, 3.48)


설치 슬롯  Level1  (-7.5, 0.0) (-2.5, 2.5) ( 2.5, 2.5) ( 7.5, 0.0)
           Level2  (-7.5, 0.0) (-2.5, 3.36) ( 2.5, 3.36) ( 7.5, 0.0)
```

부록 B. 함께 제공하는 자료

자료	내용
Doxygen HTML 문서	스크립트 8개에 한국어 Doxygen 주석을 붙이고 HTML 로 생성한 것. 클래스 다이어그램, 호출 관계, 소스 브라우저, 버그 목록 페이지를 포함한다. 원본 코드는 한 글자도 바뀌지 않았다 — 주석을 제거하면 원본과 바이트 단위로 같다 (8/8 검증 완료).
주석이 달린 .cs 8개	748줄 → 1,764줄. Unity 프로젝트에 그대로 넣어도 동작한다. 원본의 인도네시아어 주석도 지우지 않고 보존했고, 그 아래에 한국어 번역을 덧붙였다.
이 GDD 문서	.docx

Doxygen 문서는 index.html 을 브라우저로 열면 된다. 왼쪽 트리에서 클래스를 고르면 멤버별 설명과 소스가 나오고, "버그 목록" 페이지에 재현 경로가 확인된 결함이 모여 있다.